



**SCTR'S PUNE INSTITUTE OF COMPUTER
TECHNOLOGY, PUNE
DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND
DATA SCIENCE**

2025-2026

**Demonstration of Stored Cross-Site Scripting (XSS)
Vulnerability Using a Full-Stack Blog Application**

Submitted to the

Savitribai Phule Pune University

In partial fulfillment for the award of the Degree of

Bachelor of Engineering

in

Artificial Intelligence & Data Science

by

Vivek Rajendra Bhalke

(Roll No. 34107)

Under the guidance of

Prof. Anjali Deshpande

Contents

ABSTRACT	3
1 INTRODUCTION	4
1.1 Overview of Web Security	4
1.2 What is Cross-Site Scripting (XSS)?	4
1.3 Why Stored XSS is the Most Dangerous	5
1.4 Objective of this Mini Project	5
2 PROBLEM STATEMENT	6
2.1 Problem Definition	6
2.2 Specific Challenges	6
2.2.1 Server-Side Vulnerability	6
2.2.2 Client-Side Vulnerability	6
2.2.3 Browser-Level Mitigations and Bypass	7
2.3 Success Criteria	7
3 METHODOLOGY	8
3.1 System Architecture	8
3.2 Technologies Used	9
3.3 Implementation Flow	9
3.3.1 Phase 1: Backend Setup	9
3.3.2 Phase 2: Frontend Development	10
3.3.3 Phase 3: Attack Demonstration	10
3.3.4 Phase 4: The Fix	11
4 OUTCOME	13
4.1 Results	13
4.2 Screenshots of the Application	13
4.3 Observations	16
5 CONCLUSION	17
5.1 Summary	17

ABSTRACT

This project demonstrates a Stored Cross-Site Scripting (XSS) vulnerability in a real, working full-stack web application built for educational and cybersecurity awareness purposes. A complete blog website was developed using Next.js on the frontend and Express.js with Prisma ORM and PostgreSQL on the backend to simulate a production-grade environment. The application intentionally omits input sanitization on the server side and output encoding on the client side, replicating the conditions found in real-world vulnerable systems. An attacker can submit a malicious HTML payload as a blog comment — specifically `` — which is stored verbatim in the PostgreSQL database without any filtering. When any innocent user subsequently visits the same blog post, the payload executes automatically in their browser without requiring any interaction on their part. The `onerror` event handler vector is used because modern browsers block `<script>` tag injection via `innerHTML` as a shallow browser-level mitigation — this project demonstrates that such mitigations are insufficient, as attackers always discover alternate execution vectors. The project also presents the one-line fix: replacing `dangerouslySetInnerHTML` with plain JSX text rendering in React, which causes the browser to escape all HTML and render the payload as harmless text. The overarching goal is to build a thorough understanding of the mechanics of stored XSS, its real-world impact on unsuspecting users, and the layered approach required to prevent it effectively.

Chapter 1 INTRODUCTION

1.1 Overview of Web Security

As web applications have grown increasingly complex and user-interactive, security vulnerabilities have emerged as one of the most critical concerns in modern software development. Contemporary web platforms allow users to submit, store, and share content dynamically — and this interactivity, while powerful, introduces significant attack surfaces. Among the most well-known categories of web vulnerabilities are **injection attacks**, where malicious input provided by a user is interpreted as executable code by either the browser or the server. These attacks exploit the fundamental trust a system places in user-supplied data.

The **Open Web Application Security Project (OWASP)**, a globally recognized authority on web application security, consistently lists Cross-Site Scripting (XSS) and injection-based vulnerabilities in its Top 10 most critical web application security risks. According to OWASP's 2021 edition, injection attacks — including XSS — represent one of the most prevalent and damaging classes of vulnerabilities affecting web applications worldwide. Despite being a well-documented problem for over two decades, XSS remains widespread because developers are often aware of it theoretically but fail to apply preventive measures consistently in practice.

1.2 What is Cross-Site Scripting (XSS)?

Cross-Site Scripting (XSS) is a class of security vulnerability that allows an attacker to inject malicious client-side scripts — typically JavaScript — into web pages that are subsequently viewed by other users. The fundamental problem is that the browser cannot distinguish between legitimate scripts loaded by the developer and injected scripts introduced by an attacker. As a result, the browser executes both with equal trust and privilege.

There are three primary types of XSS:

- **Reflected XSS:** The malicious payload is embedded in a crafted URL. It is reflected off the web server — for example, in a search result — and executed in the browser of the user who clicks the link. It affects only that single request and requires the victim to click a suspicious URL.
- **Stored XSS:** The malicious payload is submitted by the attacker and permanently saved in the server's database. Every user who loads the affected page subsequently receives

and executes the payload. No special link is needed — simply visiting the page triggers the attack.

- **DOM-based XSS:** The vulnerability exists entirely on the client side. The payload manipulates the Document Object Model (DOM) using JavaScript, without any server involvement. The server may be completely secure, but the frontend code handles untrusted data unsafely.

1.3 Why Stored XSS is the Most Dangerous

Stored XSS is considered the most dangerous variant because the malicious script is **permanently saved in the server's database** and executes for every single user who visits the affected page — automatically, without any action required from the victim. Unlike reflected XSS, which requires a victim to click a crafted link, stored XSS is entirely passive from the victim's perspective. Visiting a familiar, trusted website is sufficient to trigger the attack.

The real-world consequences of a successful stored XSS attack can be severe. Attackers can use the injected script to steal session cookies and hijack authenticated user accounts, redirect users to phishing websites or malware download pages, log keystrokes and capture passwords or credit card numbers, deface the website to damage organizational reputation, or silently exfiltrate sensitive data visible on the page. Because every visitor becomes a victim, a single stored XSS injection in a high-traffic application can compromise thousands of user accounts within minutes.

1.4 Objective of this Mini Project

The objective of this project is to build a **realistic, working web application** — a fully functional blog platform — that contains an intentional stored XSS vulnerability, demonstrate the complete attack lifecycle in a controlled environment from initial injection to execution in a victim's browser, and conclusively show the fix. Unlike abstract textbook descriptions of XSS, this project operationalizes the concept through a real Next.js and Express.js application connected to a live PostgreSQL database.

The project bridges the critical gap between theoretical understanding and practical implementation of web security concepts. By experiencing both the attacker's perspective (submitting the payload) and the victim's perspective (having a script execute automatically upon page visit), the project builds intuition that goes beyond what can be conveyed by reading alone.

Chapter 2 PROBLEM STATEMENT

2.1 Problem Definition

Many developers are aware of Cross-Site Scripting theoretically but do not fully understand how it manifests in a real application built with modern frameworks. Tutorials and textbooks explain it abstractly, often with simplified toy examples that do not reflect the architecture of a real project. This project addresses that gap by building a complete, working vulnerable application from scratch using industry-standard tools — Next.js, Express.js, Prisma, and PostgreSQL — and demonstrating the full attack lifecycle in a live, running environment.

The core problem being addressed is: *How does stored XSS actually work end-to-end in a modern full-stack web application, and what does the fix look like in practice?* The answer requires understanding both the server-side storage flaw and the client-side rendering flaw simultaneously, since both are required for the vulnerability to be exploitable.

2.2 Specific Challenges

2.2.1 Server-Side Vulnerability

The Express.js backend receives comment data submitted by users through the frontend form and stores it directly in the PostgreSQL database via Prisma ORM, **without any sanitization or validation of HTML content**. This means that any HTML tags, JavaScript code, or event handlers submitted by a user are stored in the database verbatim, exactly as typed. There is no use of any sanitization library (such as DOMPurify or sanitize-html) on the server, no rejection of inputs containing HTML special characters, and no content-type enforcement. The raw string is persisted to the `comment` column of the `Comment` table without modification.

2.2.2 Client-Side Vulnerability

The Next.js frontend fetches comments from the backend API and renders them using React's **`dangerouslySetInnerHTML`** property. This property is React's explicit mechanism for injecting raw HTML strings directly into the DOM — bypassing React's standard, automatic HTML escaping. React's default behavior (rendering text as JSX children) would safely escape all HTML. By explicitly choosing `dangerouslySetInnerHTML`, the developer instructs React to treat the string as trusted HTML and inject it without modification. Any malicious

HTML or JavaScript stored in the database is therefore passed directly to the browser's HTML parser, which executes it.

2.2.3 Browser-Level Mitigations and Bypass

Modern browsers implement a shallow mitigation: `<script>` tags injected via `innerHTML` are not executed. This is a browser-level safeguard, not a fix for the underlying vulnerability. The project demonstrates that this mitigation is trivially bypassed using **HTML event handler vectors**. For example, the payload `` works because it does not use a `<script>` tag at all. The browser parses the `<img>` element, attempts to load the image from the source `"x"`, fails (since no such image exists), and immediately fires the `onerror` event handler — executing the JavaScript within it. Other similar vectors include `<svg onload="alert()" ">` and `<body onpageshow="alert()" ">`. Blocking `<script>` tags provides false security; the underlying `innerHTML` injection itself must be eliminated.

2.3 Success Criteria

The project is considered fully successful when the following conditions are all met:

1. A malicious comment payload is successfully submitted through the comment form without any error or rejection.
2. The payload is stored in the PostgreSQL database without any modification — confirmed by the raw string being present in the `comment` column.
3. When a different user (in a separate browser session or incognito window) visits the same blog post, the payload executes **automatically** — without any interaction required from the victim.
4. The browser alert dialog displaying `"XSS Attacked!"` fires immediately upon page load in the victim's browser.
5. The second blog post ("The Art of Slow Living") remains completely unaffected, proving that the `blogId`-based comment isolation is working correctly.
6. The fix — replacing `dangerouslySetInnerHTML` with plain JSX text rendering — is demonstrated to completely neutralize the attack, causing the payload to render as harmless visible text.

Chapter 3 METHODOLOGY

3.1 System Architecture

The project follows a classic two-tier **client-server architecture** with a clear separation between the presentation layer and the data layer, connected via a RESTful HTTP API.

- **Frontend:** A Next.js 15 App Router application running on `localhost:3000`. It handles all UI rendering, routing between pages, and communication with the backend.
- **Backend:** An Express.js REST API server running on `localhost:8000`. It receives comment submissions, interacts with the database, and serves comment data.
- **Database:** PostgreSQL, accessed through Prisma ORM. It persistently stores all comments submitted through the frontend.
- **Communication:** The frontend communicates with the backend via HTTP `fetch` calls. CORS (Cross-Origin Resource Sharing) is configured on the Express.js server to accept requests from `http://localhost:3000`.
- **Blog Content:** The blog post content (titles, slugs, body text) is hardcoded on the frontend in a file called `lib/blogData.js`. Only comments are dynamic and database-driven.

3.2 Technologies Used

Technology	Purpose	Version / Details
Next.js	Frontend framework	v15, App Router, TypeScript
Tailwind CSS	UI styling	Custom glassmorphism utilities
Express.js	Backend REST API	Node.js based server
Prisma ORM	Database access layer	Type-safe queries
PostgreSQL	Relational database	Stores comments persistently
Node.js	Runtime environment	ES Modules
dangerouslySetInnerHTML	Internal XSS vector	Renders raw HTML into DOM

3.3 Implementation Flow

3.3.1 Phase 1: Backend Setup

The Express.js server exposes two API routes for comment management:

- **GET** `/api/comments?blogId={id}`: Fetches all comments for a specific blog post filtered by `blogId`, returned in descending order of `createdAt` so the most recent comment appears first.
- **POST** `/api/comment`: Accepts a JSON body containing `{blogId, name, comment}`, validates that all three fields are present, and stores the comment in PostgreSQL via Prisma **with zero sanitization**.

The Prisma schema for the `Comment` model is defined as follows:

```
1 model Comment {
2   id          Int          @id @default(autoincrement())
3   blogId      String       // Identifies the blog post (slug)
4   name        String       // Commenter's name
5   comment     String       // Raw input -- Vulnerable Field
6   createdAt   DateTime     @default(now())
7   updatedAt   DateTime     @updatedAt
8 }
```

Listing 3.1: Prisma Comment Model (schema.prisma)

The `blogId` field is a critical design decision. It stores the slug of the blog post (e.g., `rise-of-ai` or `art-of-slow-living`) and is used to filter comments per post. This means an XSS payload injected on one blog post only affects visitors of that specific post — it does not affect other posts, which is demonstrated as part of the success criteria.

3.3.2 Phase 2: Frontend Development

Two pages were built:

Homepage (`app/page.tsx`): Displays a blog landing page with a glassmorphism hero section containing the heading “Thoughts, Stories & Ideas” and a sticky navigation bar. Below the hero, a two-column grid renders blog post cards for the two hardcoded posts: “The Rise of Artificial Intelligence” (slug: `rise-of-ai`) and “The Art of Slow Living” (slug: `art-of-slow-living`). Blog data is sourced from `lib/blogData.js`.

Blog Post Page (`app/blog/[slug]/page.tsx`): A dynamic page that reads the `slug` parameter from the URL, finds the matching blog post from `blogData.js`, and renders the full article. Below the article, it fetches comments from the backend at `/api/comments?blogId={slug}` and renders them. The critical vulnerability is in the comment rendering code:

```
1 { /* VULNERABLE -- Raw HTML injected directly into the DOM */  
2 <div  
3   dangerouslySetInnerHTML={{ __html: comment.comment }}  
4 />
```

Listing 3.2: Vulnerable Comment Rendering (page.tsx)

By using `dangerouslySetInnerHTML`, the developer bypasses React’s automatic HTML escaping. Any raw HTML or JavaScript stored in the database field `comment.comment` is injected directly into the browser’s DOM and executed.

3.3.3 Phase 3: Attack Demonstration

The end-to-end attack proceeds through the following steps:

1. **Attacker Action:** The attacker opens the “Rise of Artificial Intelligence” blog post and submits a comment using the Leave a Comment form with the following payload in the comment field:

```
1   
2
```

Listing 3.3: XSS Attack Payload

2. **Storage:** The Express.js backend receives the payload and passes it directly to Prisma's `create` method. It is stored in the PostgreSQL `comment` column as a raw string — no sanitization occurs at any point.
3. **Victim Visits:** A completely different user (or the same user in a separate incognito browser window) opens the same blog post URL.
4. **Rendering:** The frontend fetches the stored comments and renders each one via `dangerouslySetInnerHTML`. The browser receives the raw HTML string `` and parses it as live HTML.
5. **Execution:** The browser creates an `<img>` element, attempts to load an image from the source `"x"`. This fails immediately since no such resource exists. The browser fires the `onerror` event handler, executing the JavaScript `alert('XSS Attacked!')` in the victim's browser context.
6. **Impact:** The victim sees a browser alert dialog displaying “XSS Attacked!” — they performed no suspicious action and were unaware of any attack. Their browser's session cookies, local storage, and DOM data are accessible to any more sophisticated payload in place of the benign alert.

3.3.4 Phase 4: The Fix

The fix requires a single-line change on the frontend. The vulnerable rendering code is replaced as follows:

```
1 // VULNERABLE (before)
2 <div dangerouslySetInnerHTML={{ __html: comment.comment }} />
3
4 // FIXED (after)
5 <div>{comment.comment}</div>
```

Listing 3.4: Before and After – The Fix

When React renders text using JSX curly braces `{comment.comment}`, it automatically **escapes all HTML special characters** before injecting content into the DOM. Specifically, `<` is converted to `<`, `>` to `>`, `"` to `"`, and `&` to `&`. As a result, the payload `` is displayed as literal visible text on the screen and is never interpreted as HTML by the browser.

For complete protection, a **server-side fix** is also required: using a library such as `DOMPurify` or `sanitize-html` to strip or escape HTML tags from user input before storing it in the

database. This ensures that even if a future developer inadvertently reintroduces `dangerouslySetInnerHTML` the stored data is already safe.

Chapter 4 OUTCOME

The project was successfully implemented and demonstrated end-to-end. All components — the Next.js frontend, Express.js backend, Prisma ORM, and PostgreSQL database — were integrated into a functioning application. The full attack lifecycle was executed in a live environment, and the fix was validated.

4.1 Results

All defined success criteria were met:

- The malicious payload `` was stored verbatim in the PostgreSQL `comment` table without any modification.
- The XSS alert fired automatically in the victim's browser upon page load — without any interaction from the victim beyond visiting the blog post URL.
- The second blog post (“The Art of Slow Living”) remained completely clean and unaffected, confirming that `blogId`-based comment isolation works correctly.
- The frontend fix — replacing `dangerouslySetInnerHTML` with plain `JSX {comment.comment}` rendering — successfully neutralized the attack. The payload was rendered as harmless visible text in the browser instead of executing as code.

4.2 Screenshots of the Application

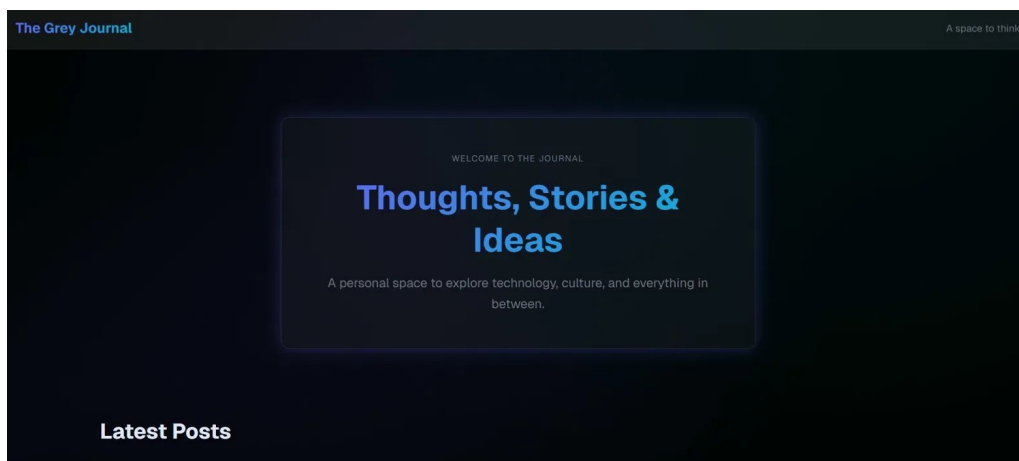


Figure 4.1: Homepage — Blog Landing Page

The homepage of “The Grey Journal” blog displays a dark-themed glassmorphism hero section with the heading “Thoughts, Stories & Ideas” and a sticky navigation bar. This is the entry point to the application from which users navigate to individual blog posts.

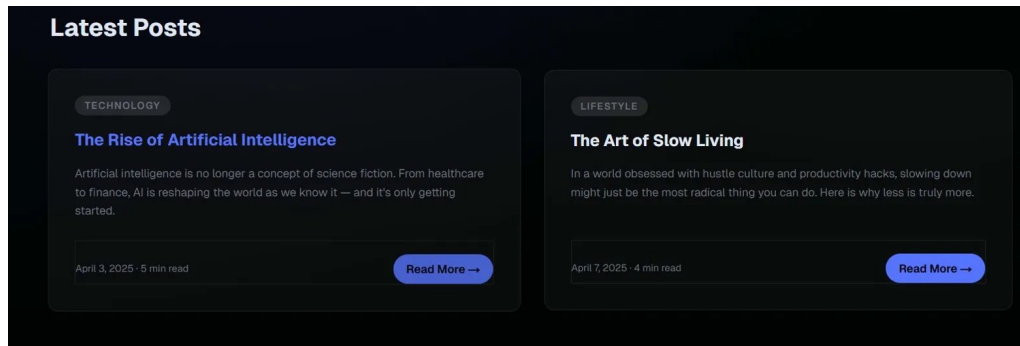


Figure 4.2: Blog Post Cards Grid — Latest Posts Section

The Latest Posts section shows two blog post cards in a two-column grid layout. The left card, “The Rise of Artificial Intelligence” (Technology), is the target post used in the XSS demonstration. The right card, “The Art of Slow Living” (Lifestyle), serves as the control post to verify attack isolation.

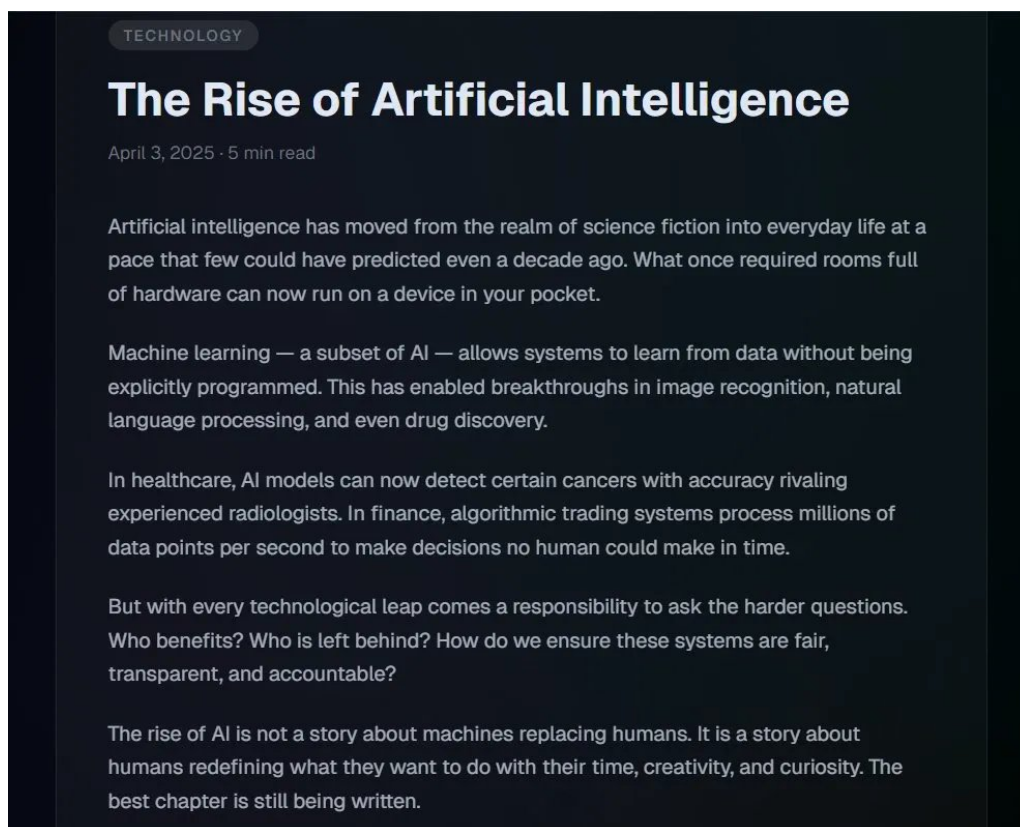


Figure 4.3: Blog Post Content Page — Article Rendering

The full blog post page for “The Rise of Artificial Intelligence” shows the complete article

content rendered inside a dark glassmorphism card. The comment section and the vulnerable comment rendering code are located below this article content.

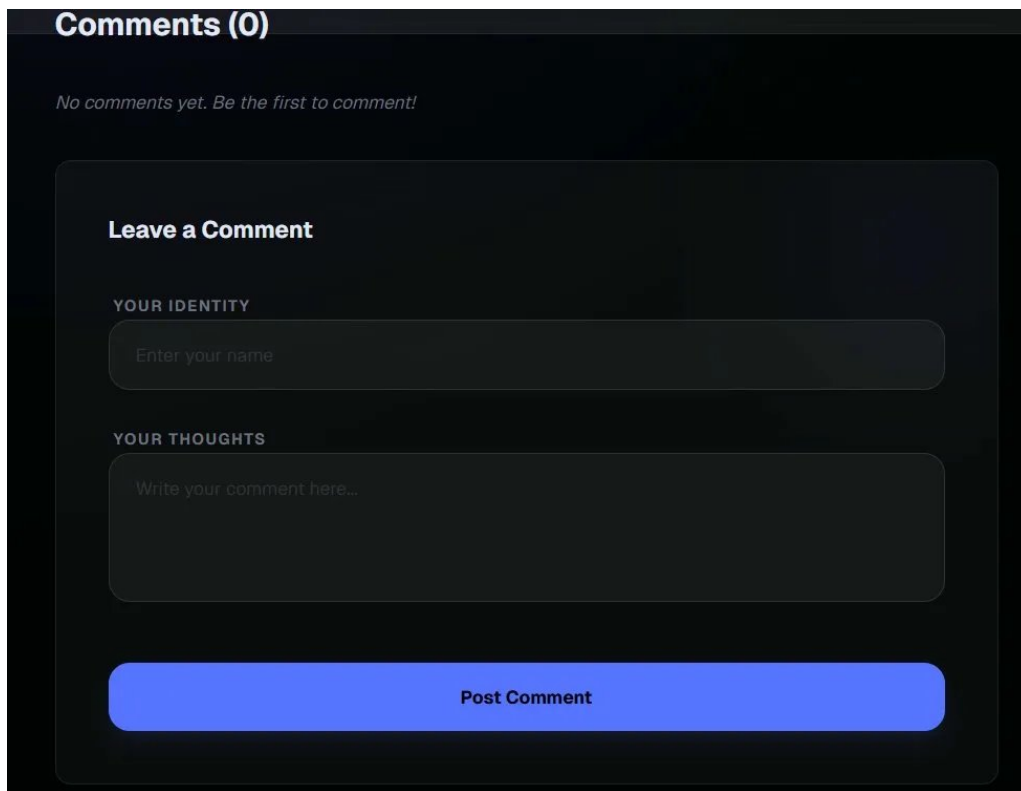
A screenshot of a web application's comment section. At the top, it says 'Comments (0)' in bold. Below that, a message reads 'No comments yet. Be the first to comment!'. The main section is titled 'Leave a Comment'. It contains two input fields: the first is labeled 'YOUR IDENTITY' with a placeholder 'Enter your name'; the second is labeled 'YOUR THOUGHTS' with a placeholder 'Write your comment here...'. At the bottom of the form is a blue button labeled 'Post Comment'.

Figure 4.4: Comment Form — Before Attack

The comment section shows zero comments with the message “No comments yet. Be the first to comment!” and the Leave a Comment form with fields for name (“YOUR IDENTITY”) and comment body (“YOUR THOUGHTS”), followed by a Post Comment button. This is the form used by the attacker to submit the malicious payload.

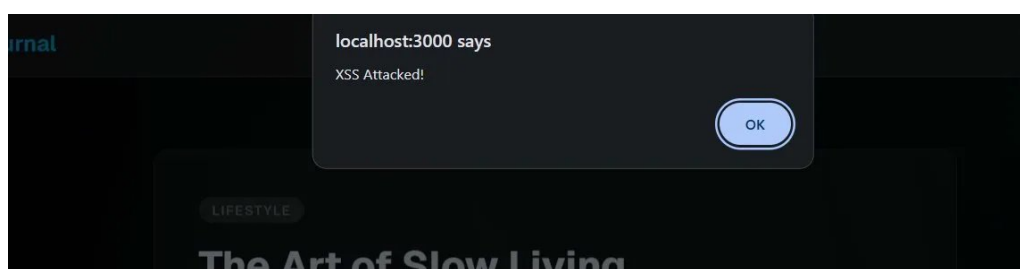


Figure 4.5: Stored XSS Attack — Alert Executed in Victim’s Browser

The browser alert dialog displaying “XSS Attacked!” fires automatically from `localhost:3000` when the victim opens the blog post page. The victim performed no suspicious action — the script executed entirely due to the stored payload being rendered via `dangerouslySetInnerHTML`.

4.3 Observations

1. The `<script>` tag, when injected via `dangerouslySetInnerHTML` in a previous test, was silently blocked by the browser. This confirms that the browser’s script-tag blocking is a shallow, browser-level mitigation and is **not a fix** for the underlying vulnerability at the application layer.
2. The `<img onerror>` event handler vector successfully bypassed the browser’s script-tag blocking and executed the JavaScript payload. This demonstrates that the root vulnerability — raw HTML injection via `dangerouslySetInnerHTML` — was never addressed by the browser’s shallow mitigation. Only one specific vector was blocked while the vulnerability itself remained fully exploitable.
3. The `blogId` field in the database correctly isolated comments between the two blog posts. Injecting the payload on the “Rise of Artificial Intelligence” post did not trigger any alert on the “Art of Slow Living” post, confirming that the separation logic works as designed.
4. The single-line change from `dangerouslySetInnerHTML={{ __html: comment.comment }}` to `{comment.comment}` completely neutralized the attack. The payload `` was rendered as visible literal text in the browser and never executed. This demonstrates the effectiveness of React’s automatic HTML escaping.
5. The malicious payload remained stored in the PostgreSQL database even after the frontend fix was applied. This confirms that **server-side sanitization is independently necessary** — a frontend fix alone leaves the dangerous string in the database, meaning any future regression on the frontend (such as accidentally reintroducing `dangerouslySetInnerHTML`) would immediately re-expose the vulnerability.

Chapter 5 CONCLUSION

5.1 Summary

This project successfully demonstrated a Stored Cross-Site Scripting vulnerability from end to end in a modern full-stack web application built with industry-standard technologies. We established that XSS is not merely a theoretical concept — it can be implemented and exploited in a real Next.js and Express.js application with minimal effort when input sanitization is absent from the server and output encoding is absent from the client.

Through this project, we developed a thorough understanding of the three types of XSS and why stored XSS is the most dangerous variant: the payload persists permanently in the database and executes automatically for every visitor, requiring no interaction whatsoever from the victim. A single injection by one attacker can compromise every user who visits the affected page from that point forward.

We validated experimentally that browser-level mitigations — such as blocking `<script>` tag execution via `innerHTML` — are insufficient and provide false security. Attackers have a broad arsenal of alternate vectors through HTML event handlers (`onerror`, `onload`, `onclick`, etc.) that do not rely on `<script>` tags at all. The browser's shallow mitigation addresses a symptom, not the cause.

We demonstrated that the **fix must be applied at two independent levels**: on the client side by rendering untrusted content as plain text rather than raw HTML (never using `dangerouslySetInnerHTML` for user-generated content), and on the server side by sanitizing input using a trusted library such as DOMPurify before persistence. Defence in depth — applying both fixes simultaneously — is the only approach that provides comprehensive protection.

This project provided practical, hands-on exposure to an OWASP Top 10 vulnerability category and cultivated the dual mindset required in cybersecurity: thinking like an attacker to understand how systems can be exploited, and thinking like a defender to design systems that are resilient against those exploits.

REFERENCES

1. OWASP Foundation. (2021). *OWASP Top Ten — A03:2021 Injection*. Retrieved from https://owasp.org/Top10/A03_2021-Injection/
2. OWASP Foundation. *Cross Site Scripting (XSS)*. OWASP Cheat Sheet Series. Retrieved from <https://owasp.org/www-community/attacks/xss/>
3. MDN Web Docs. (2024). *Cross-site scripting (XSS)*. Mozilla Developer Network. Retrieved from https://developer.mozilla.org/en-US/docs/Glossary/Cross-site_scripting
4. React Documentation. (2024). *dangerouslySetInnerHTML*. React Official Docs. Retrieved from <https://react.dev/reference/react-dom/components/common#dangerously-setting-the-inner-html>
5. Prisma. (2024). *Prisma ORM Documentation*. Retrieved from <https://www.prisma.io/docs>
6. Express.js. (2024). *Express.js Official Documentation*. Retrieved from <https://expressjs.com/>
7. Next.js. (2024). *Next.js App Router Documentation*. Vercel. Retrieved from <https://nextjs.org/docs>